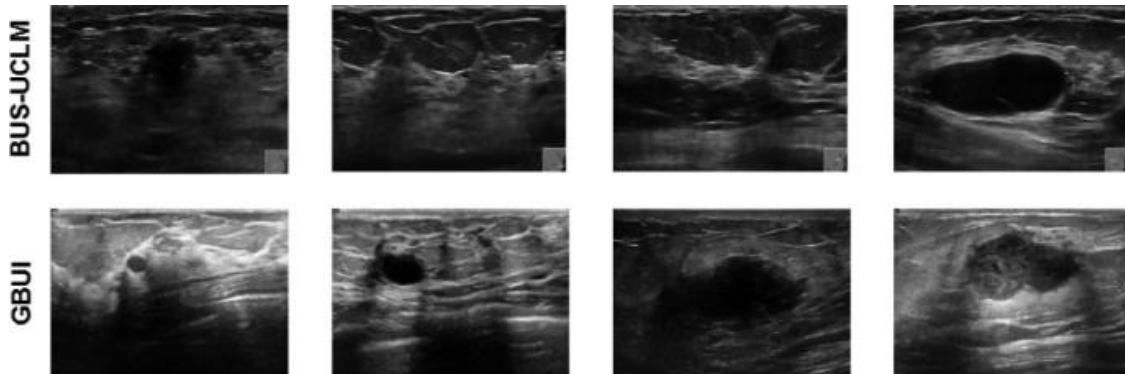


Brain Tumor Classification using ScaleNet Architecture



```
import numpy as np
import pandas as pd
import os

base_path = "/kaggle/input/bus-uclm-breast-ultrasound-
dataset/bus_uclm_separated/"
categories = ["benign", "malign", "normal"]

image_paths = []
labels = []

for category in categories:
    category_path = os.path.join(base_path, category)
    for image_name in os.listdir(category_path):
        image_path = os.path.join(category_path, image_name)
        image_paths.append(image_path)
        labels.append(category)

df = pd.DataFrame({
    "image_path": image_paths,
    "label": labels
})

df.head()

          image_path  label
0  /kaggle/input/bus-uclm-breast-ultrasound-datas...  benign
1  /kaggle/input/bus-uclm-breast-ultrasound-datas...  benign
2  /kaggle/input/bus-uclm-breast-ultrasound-datas...  benign
3  /kaggle/input/bus-uclm-breast-ultrasound-datas...  benign
4  /kaggle/input/bus-uclm-breast-ultrasound-datas...  benign

df.tail()

          image_path  label
678  /kaggle/input/bus-uclm-breast-ultrasound-datas...  normal
```

```
679 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
680 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
681 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
682 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
```

```
df.shape
```

```
(683, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
label         0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 683 entries, 0 to 682
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   image_path  683 non-null   object
 1   label      683 non-null   object
dtypes: object(2)
memory usage: 10.8+ KB
```

```
df['label'].unique()
```

```
array(['benign', 'malign', 'normal'], dtype=object)
```

```
df['label'].value_counts()
```

```
label
normal    419
benign    174
malign     90
Name: count, dtype: int64
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.set_style("whitegrid")
```

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)
```

```

ax.set_title("Distribution Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

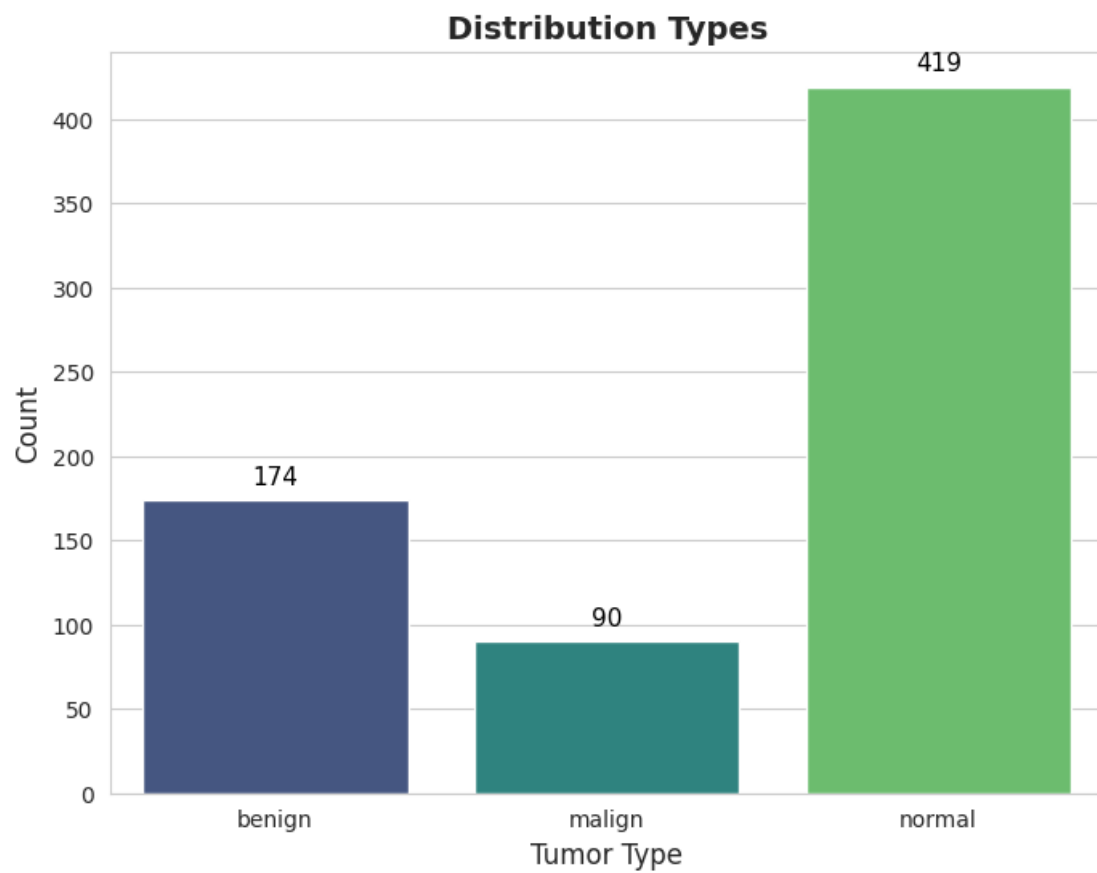
fig, ax = plt.subplots(figsize=(8, 6))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 12, 'weight':
        'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

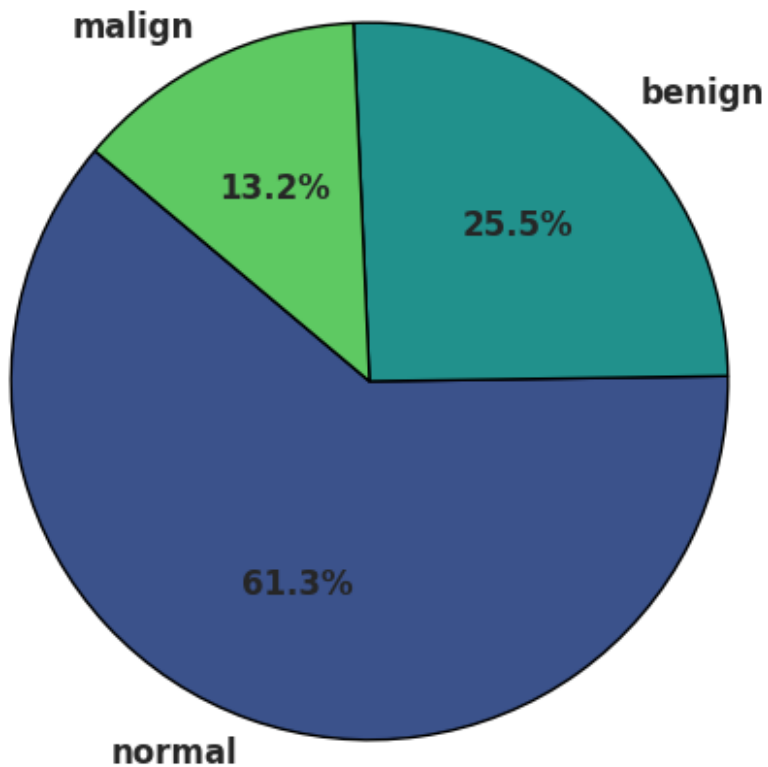
ax.set_title("Distribution Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



Distribution Types - Pie Chart



```
import cv2

num_images = 5

plt.figure(figsize=(15, 12))

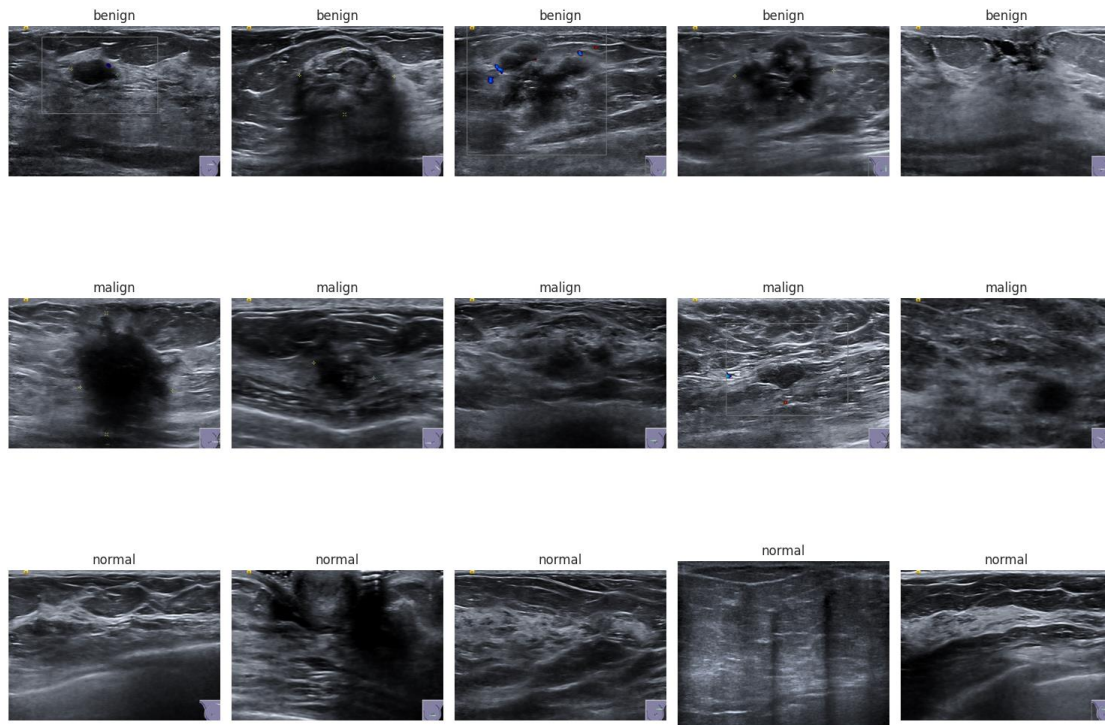
for i, category in enumerate(categories):
    category_images = df[df['label'] ==
category]['image_path'].iloc[:num_images]

    for j, img_path in enumerate(category_images):

        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.subplot(len(categories), num_images, i * num_images + j + 1)
        plt.imshow(img)
        plt.axis('off')
        plt.title(category)

plt.tight_layout()
plt.show()
```



```
from sklearn.utils import resample
```

```
max_count = df['label'].value_counts().max()
```

```
dfs = []
```

```
for category in df['label'].unique():
    class_subset = df[df['label'] == category]
    class_upsampled = resample(class_subset,
                              replace=True,
                              n_samples=max_count,
                              random_state=42)
    dfs.append(class_upsampled)
```

```
df_balanced = pd.concat(dfs).sample(frac=1,
                                     random_state=42).reset_index(drop=True)
```

```
df = df_balanced
```

```
df
```

	image_path	label
0	/kaggle/input/bus-uclm-breast-ultrasound-datas...	malign
1	/kaggle/input/bus-uclm-breast-ultrasound-datas...	benign
2	/kaggle/input/bus-uclm-breast-ultrasound-datas...	benign
3	/kaggle/input/bus-uclm-breast-ultrasound-datas...	benign
4	/kaggle/input/bus-uclm-breast-ultrasound-datas...	normal
...	...	...
1252	/kaggle/input/bus-uclm-breast-ultrasound-datas...	normal
1253	/kaggle/input/bus-uclm-breast-ultrasound-datas...	normal
1254	/kaggle/input/bus-uclm-breast-ultrasound-datas...	normal

```
1255 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
1256 /kaggle/input/bus-uclm-breast-ultrasound-datas... normal
```

```
[1257 rows x 2 columns]
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from timm import create_model
from sklearn.model_selection import train_test_split
from PIL import Image
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.cuda.amp import autocast, GradScaler
from torch.utils.data.sampler import WeightedRandomSampler
import os

class UltrasoundDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe
        self.transform = transform
        self.label_map = {'malign': 0, 'benign': 1, 'normal': 2}

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx]['image_path']
        label = self.dataframe.iloc[idx]['label']
        label = self.label_map[label]
        try:
            image = Image.open(img_path).convert('RGB')
        except FileNotFoundError:
            image = Image.fromarray(np.zeros((224, 224, 3), dtype=np.uint8))
        if self.transform:
            image = self.transform(image)
        return image, label

class RandomCutmixOrMixup(nn.Module):
    def __init__(self, p=0.4, mixup_alpha=0.2, cutmix_alpha=1.0,
num_classes=3):
        super().__init__()
        self.p = p
        self.mixup_alpha = mixup_alpha
```

```

self.cutmix_alpha = cutmix_alpha
self.num_classes = num_classes

def forward(self, x, y):
    if torch.rand(1) >= self.p:
        return x, y
    batch_size = x.size(0)
    lam = np.random.beta(self.mixup_alpha, self.mixup_alpha)
    if torch.rand(1) < 0.5:
        rand_index = torch.randperm(batch_size).to(x.device)
        x = lam * x + (1 - lam) * x[rand_index, :]
        y_onehot = F.one_hot(y, num_classes=self.num_classes).float()
        y = lam * y_onehot + (1 - lam) * y_onehot[rand_index]
    else:
        rand_index = torch.randperm(batch_size).to(x.device)
        W, H = x.shape[2:]
        r_x = np.random.uniform(W)
        r_y = np.random.uniform(H)
        r_w = W * np.sqrt(1 - lam)
        r_h = H * np.sqrt(1 - lam)
        x1 = np.clip(r_x - r_w / 2, 0, W).astype(int)
        y1 = np.clip(r_y - r_h / 2, 0, H).astype(int)
        x2 = np.clip(r_x + r_w / 2, 0, W).astype(int)
        y2 = np.clip(r_y + r_h / 2, 0, H).astype(int)
        x[:, :, y1:y2, x1:x2] = x[rand_index, :, y1:y2, x1:x2]
        lam = 1 - ((x2 - x1) * (y2 - y1) / (W * H))
        y_onehot = F.one_hot(y, num_classes=self.num_classes).float()
        y = lam * y_onehot + (1 - lam) * F.one_hot(y[rand_index],
num_classes=self.num_classes).float()
        return x, y

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.1,
hue=0.05),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9,
1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

train_df_full, test_df = train_test_split(df, test_size=0.2, random_state=42,

```

```

stratify=df['label'])
train_df, val_df = train_test_split(train_df_full, test_size=0.125,
random_state=42, stratify=train_df_full['label'])

train_dataset = UltrasoundDataset(train_df, transform=train_transform)
val_dataset = UltrasoundDataset(val_df, transform=eval_transform)
test_dataset = UltrasoundDataset(test_df, transform=eval_transform)

class_counts = train_df['label'].value_counts()
class_weights = 1. / class_counts.sort_index().values
class_weights = class_weights / class_weights.sum()
sample_weights = np.array([class_weights[train_dataset.label_map[label]] for
label in train_df['label']])
sampler = WeightedRandomSampler(sample_weights, len(sample_weights),
replacement=True)

train_loader = DataLoader(train_dataset, batch_size=32, sampler=sampler)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

class ViTLayer(nn.Module):
    def __init__(self, layer):
        super(ViTLayer, self).__init__()
        self.layer = layer

    def forward(self, x):
        return self.layer(x)

class Adapter(nn.Module):
    def __init__(self, d_model, r=80):
        super(Adapter, self).__init__()
        self.down = nn.Linear(d_model, r)
        self.up = nn.Linear(r, d_model)
        self.act = nn.GELU()
        self.dropout = nn.Dropout(0.1)
        nn.init.xavier_uniform_(self.down.weight)
        nn.init.zeros_(self.up.weight)
        nn.init.zeros_(self.up.bias)

    def forward(self, x):
        return self.up(self.dropout(self.act(self.down(x))))

class ScaleNetLayer(nn.Module):
    def __init__(self, base_layer, r=80):
        super(ScaleNetLayer, self).__init__()
        self.base_layer = base_layer
        self.adapter = Adapter(768, r)
        for param in self.base_layer.parameters():
            param.requires_grad = False

    def forward(self, x):

```

```

        x_base = self.base_layer(x)
        x_adj = self.adapter(x_base)
        return x_base + x_adj

class PatchEmbed(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_chans=3,
embed_dim=768):
        super(PatchEmbed, self).__init__()
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) * (img_size //
patch_size)
        self.proj = nn.Conv2d(in_chans, embed_dim, kernel_size=patch_size,
stride=patch_size)

    def forward(self, x):
        B, C, H, W = x.shape
        x = self.proj(x)
        x = x.flatten(2).transpose(1, 2)
        return x

class ScaleNet(nn.Module):
    def __init__(self, pretrained_vit, scale_factor=2, r=80):
        super(ScaleNet, self).__init__()
        self.patch_embed = PatchEmbed()

self.patch_embed.load_state_dict(pretrained_vit.patch_embed.state_dict())
self.cls_token = nn.Parameter(pretrained_vit.cls_token.clone())
self.pos_embed = pretrained_vit.pos_embed
self.cls_token.requires_grad = False
self.pos_embed.requires_grad = False
for param in self.patch_embed.parameters():
    param.requires_grad = True
self.layers = nn.ModuleList()
self.blocks = pretrained_vit.blocks
L = len(self.blocks)
L_prime = int(L * scale_factor)
for l in range(L_prime):
    if l < L:
        self.layers.append(ViTLayer(self.blocks[l]))
        for param in self.blocks[l].parameters():
            param.requires_grad = False
    else:
        mapped_idx = l % L

self.layers.append(ScaleNetLayer(ViTLayer(self.blocks[mapped_idx]), r))
self.norm = pretrained_vit.norm
for param in self.norm.parameters():
    param.requires_grad = False
self.head = nn.Linear(pretrained_vit.head.in_features, 3)

    def forward(self, x):

```

```

x = self.patch_embed(x)
cls_tokens = self.cls_token.expand(x.shape[0], -1, -1)
x = torch.cat((cls_tokens, x), dim=1)
x = x + self.pos_embed.expand(x.shape[0], -1, -1).to(x.dtype)
for layer in self.layers:
    x = layer(x)
x = self.norm(x)
x = x[:, 0]
return self.head(x)

pretrained_vit = create_model('vit_base_patch16_224', pretrained=True)
scale_net = ScaleNet(pretrained_vit, scale_factor=2, r=80)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
scale_net = scale_net.to(device)

class WeightedCosineAnnealingLR(optim.lr_scheduler._LRScheduler):
    def __init__(self, optimizer, T_max, eta_min=5e-7, last_epoch=-1,
warmup_epochs=1):
        self.T_max = T_max
        self.eta_min = eta_min
        self.warmup_epochs = warmup_epochs
        super(WeightedCosineAnnealingLR, self).__init__(optimizer,
last_epoch)

    def get_lr(self):
        if self._step_count < self.warmup_epochs:
            return [base_lr * (self._step_count + 1) / self.warmup_epochs for
base_lr in self.base_lrs]
        elif self._step_count == self.warmup_epochs:
            return self.base_lrs
        else:
            t = self._step_count - self.warmup_epochs
            T = self.T_max - self.warmup_epochs
            return [self.eta_min + (base_lr - self.eta_min) *
(1 + np.cos(np.pi * t / T)) / 2
for base_lr in self.base_lrs]

def train_model(model, train_loader, val_loader, test_loader, epochs=15):
    class_weights_tensor = torch.tensor(class_weights,
dtype=torch.float32).to(device)
    optimizer_grouped_parameters = [
        {'params': [p for name, p in model.named_parameters() if 'adapter'
not in name and p.requires_grad], 'lr': 1e-4, 'weight_decay': 5e-4},
        {'params': [p for name, p in model.named_parameters() if 'adapter' in
name and p.requires_grad], 'lr': 5e-5, 'weight_decay': 5e-4},
    ]
    optimizer = optim.AdamW(optimizer_grouped_parameters)
    criterion_hard = nn.CrossEntropyLoss(weight=class_weights_tensor)
    scheduler = WeightedCosineAnnealingLR(optimizer, T_max=epochs,
warmup_epochs=1)
    scaler = GradScaler()
    mixup_cutmix = RandomCutmixOrMixup(p=0.4, mixup_alpha=0.2,

```

```

cutmix_alpha=1.0, num_classes=3)
best_val_loss = float('inf')
accumulation_steps = 8
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    optimizer.zero_grad()
    for i, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)
        images, labels_mixed = mixup_cutmix(images, labels)
        with autocast():
            outputs = model(images)
            if labels_mixed.dim() == 1:
                loss = criterion_hard(outputs, labels_mixed)
            else:
                loss = -torch.sum(F.log_softmax(outputs, dim=1) *
labels_mixed, dim=1).mean()
            scaler.scale(loss / accumulation_steps).backward()
            if (i + 1) % accumulation_steps == 0:
                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad()
                running_loss += loss.item()
            if len(train_loader) % accumulation_steps != 0:
                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad()
        val_loss, _, _ = evaluate_model(model, val_loader, criterion_hard)
        print(f"Epoch {epoch+1}, Train Loss:
{running_loss/len(train_loader):.4f}, Val Loss: {val_loss:.4f}")
        scheduler.step()
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), 'best_model.pth')
        model.load_state_dict(torch.load('best_model.pth'))
        _, all_preds, all_labels = evaluate_model(model, test_loader,
criterion_hard)
    return all_preds, all_labels

def evaluate_model(model, data_loader, criterion):
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for images, labels in data_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            all_preds.extend(predicted.cpu().numpy())

```

```

        all_labels.extend(labels.cpu().numpy())
    return total_loss / len(data_loader), all_preds, all_labels

preds, labels = train_model(scale_net, train_loader, val_loader, test_loader,
epochs=15)
overall_accuracy = accuracy_score(labels, preds)
print(f"\nOverall Test Accuracy: {overall_accuracy:.4f}")
cm = confusion_matrix(labels, preds)
label_names = ['malign', 'benign', 'normal']
print("\nConfusion Matrix (Numerical):")
print(cm)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_names,
yticklabels=label_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
report = classification_report(labels, preds, target_names=label_names,
output_dict=True)
report_df = pd.DataFrame(report).transpose()
print("\nClassification Report (Numerical):")
print(report_df)
plt.figure(figsize=(8, 6))
sns.heatmap(report_df.iloc[:-1, :-1], annot=True, cmap='Blues', fmt='.2f')
plt.title('Classification Report Metrics')
plt.show()

{"model_id": "b0159c7460ee489897ae8f37c073307a", "version_major": 2, "version_min
or": 0}

/tmp/ipykernel_36/2110194718.py:230: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use
`torch.amp.GradScaler('cuda', args...)` instead.
    scaler = GradScaler()
/tmp/ipykernel_36/2110194718.py:241: FutureWarning:
`torch.cuda.amp.autocast(args...)` is deprecated. Please use
`torch.amp.autocast('cuda', args...)` instead.
    with autocast():

Epoch 1, Train Loss: 1.2399, Val Loss: 1.0504
Epoch 2, Train Loss: 1.0543, Val Loss: 1.0347
Epoch 3, Train Loss: 0.9905, Val Loss: 0.9496
Epoch 4, Train Loss: 0.9379, Val Loss: 0.8746
Epoch 5, Train Loss: 0.8975, Val Loss: 0.8391
Epoch 6, Train Loss: 0.8774, Val Loss: 0.8247
Epoch 7, Train Loss: 0.8156, Val Loss: 0.7523
Epoch 8, Train Loss: 0.7675, Val Loss: 0.7191
Epoch 9, Train Loss: 0.7842, Val Loss: 0.7415
Epoch 10, Train Loss: 0.7149, Val Loss: 0.7413
Epoch 11, Train Loss: 0.6784, Val Loss: 0.7170
Epoch 12, Train Loss: 0.7256, Val Loss: 0.6936
Epoch 13, Train Loss: 0.7236, Val Loss: 0.6922

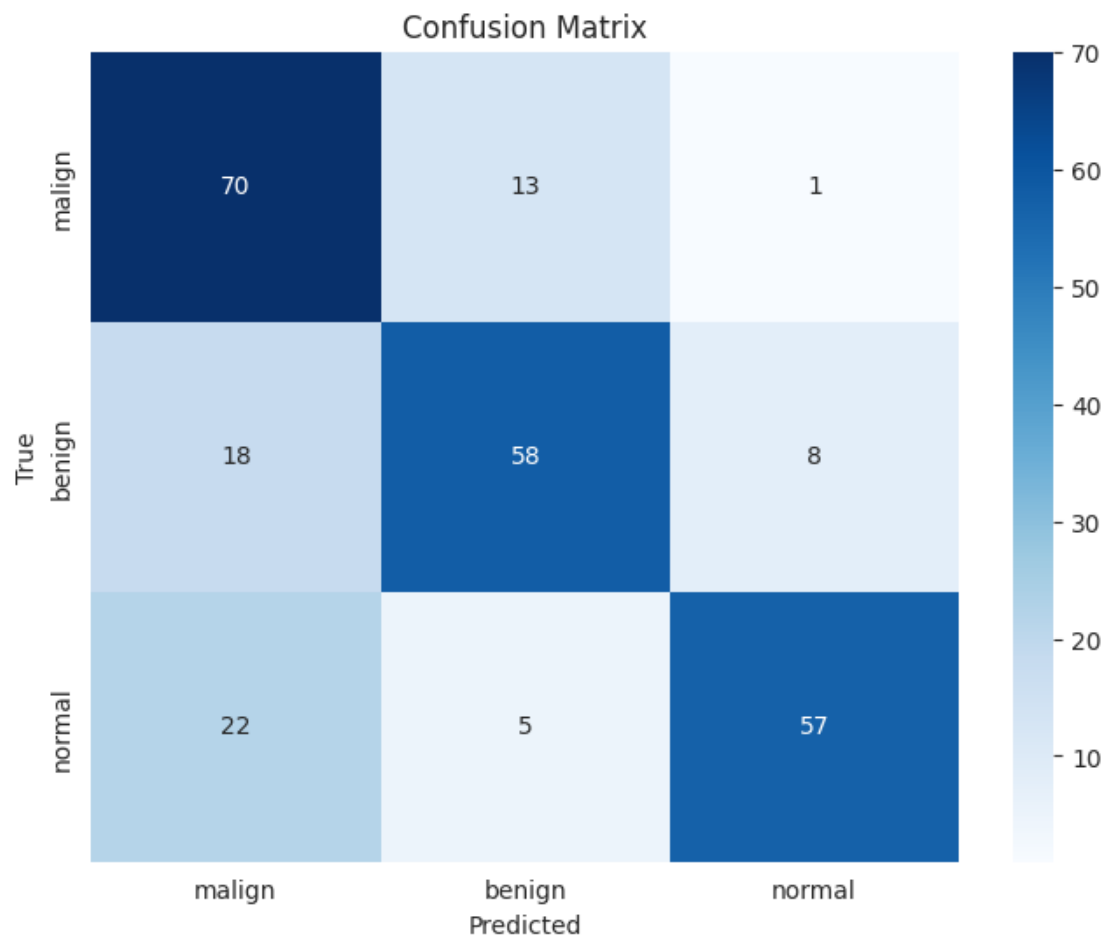
```

Epoch 14, Train Loss: 0.6540, Val Loss: 0.6931
Epoch 15, Train Loss: 0.6792, Val Loss: 0.6926

Overall Test Accuracy: 0.7341

Confusion Matrix (Numerical):

```
[[70 13  1]
 [18 58  8]
 [22  5 57]]
```



Classification Report (Numerical):

	precision	recall	f1-score	support
malign	0.636364	0.833333	0.721649	84.000000
benign	0.763158	0.690476	0.725000	84.000000
normal	0.863636	0.678571	0.760000	84.000000
accuracy	0.734127	0.734127	0.734127	0.734127
macro avg	0.754386	0.734127	0.735550	252.000000
weighted avg	0.754386	0.734127	0.735550	252.000000

